```
    command: ["--config=/conf/otel-collector-config.yaml"]
    volumes:
    - ./otel-config.yaml:/conf/otel-collector-config.yaml
    ports:
    - "8006:8006"

  clickhouse-server:
    image: clickhouse/clickhouse-server
    ports:
    - "8123:8123"
    - "9000:9000"
    ulimits:
      nproc: 65535
      nofile:
```

We need to expose the Fluent Forward receiver port here so that we can receive log lines from Docker.

This is where we set up ClickHouse: the port 8123 for the Web UI and port 9000, the default native TCP endpoint for clients (allowing the OpenTelemetry exporter to talk to it).

There are several low-level Linux settings, such as the number of processes and files allowed for ClickHouse that we're setting here.

For more details on the ClickHouse configuration and good practices, see the ClickHouse operations docs (https://clickhouse.com/docs/en/operations/settings), specifically, the excellent article on ClickHouse Networking (http://mng.bz/EQyo). Now, we will move on to the telemetry agent setup. The following listing shows the OpenTelemetry collector configuration I put together for our ClickHouse example.

Listing 5.2 OpenTelemetry collector configuration for the ClickHouse setup

```
receivers:
  fluentforward:
    endpoint: 0.0.0.0:8006

processors:
  batch:
    timeout: 5s
    send_batch_size: 100

exporters:
  logging:
    loglevel: debug
  clickhouse:
    dsn: tcp://clickhouse-server:9000/default
    ttl_days: 3

service:
  telemetry:
    logs:
      level: debug
  pipelines:
    logs:
      receivers: [ fluentforward ]
      processors: [ batch ]
      exporters: [ logging, clickhouse ]
```

We configure the Fluent Forward receiver here to listen on all addresses on port 8006 for TCP connections. If all works, the Docker Fluentd driver will connect here.

Here, we set the batch size to 100 log lines. In practice, this would be much higher, likely in the thousands (depends on how long you want to wait until the logs are accessible in ClickHouse). I chose this low number to speed up things for you to see results.

This is the place where we set up the ClickHouse exporter, configuring it with the address of where the ClickHouse server is running. From the server perspective, our exporter is just another client talking TCP with it. The ttl_days setting limits the time data is kept around (not really important for our demo).

Finally, here, we assemble the pipeline using the components we configured previously.